

Web Application Vulnerability Assessment Report

This report documents the vulnerability assessment performed on the Damn Vulnerable Web Application (DVWA) deployed in a controlled local environment. The assessment included application setup, HTTP service scanning, directory enumeration, and SQL Injection vulnerability testing.

1. Assessment Details

Target Application	Damn Vulnerable Web Application (DVWA)
Target Address	http://127.0.0.1:4280
Testing Environment	Kali Linux
Deployment Method	Docker Container
Tool Used	Metasploit Framework
Assessment Type	Web Application Vulnerability Assessment

2. Objective

The objective of this assessment was to deploy DVWA in a controlled environment and perform web application scanning and vulnerability testing. The assessment focused on identifying the HTTP service, discovering accessible directories, and testing the application for SQL Injection vulnerability.

3. DVWA Setup and Configuration

Damn Vulnerable Web Application (DVWA) was deployed locally using Docker. The application was configured successfully and the database was created and initialized.

After completing the setup process, the DVWA application was accessible through the local browser using the following address:

http://127.0.0.1:4280

Result: DVWA was successfully deployed and configured in the local testing environment.

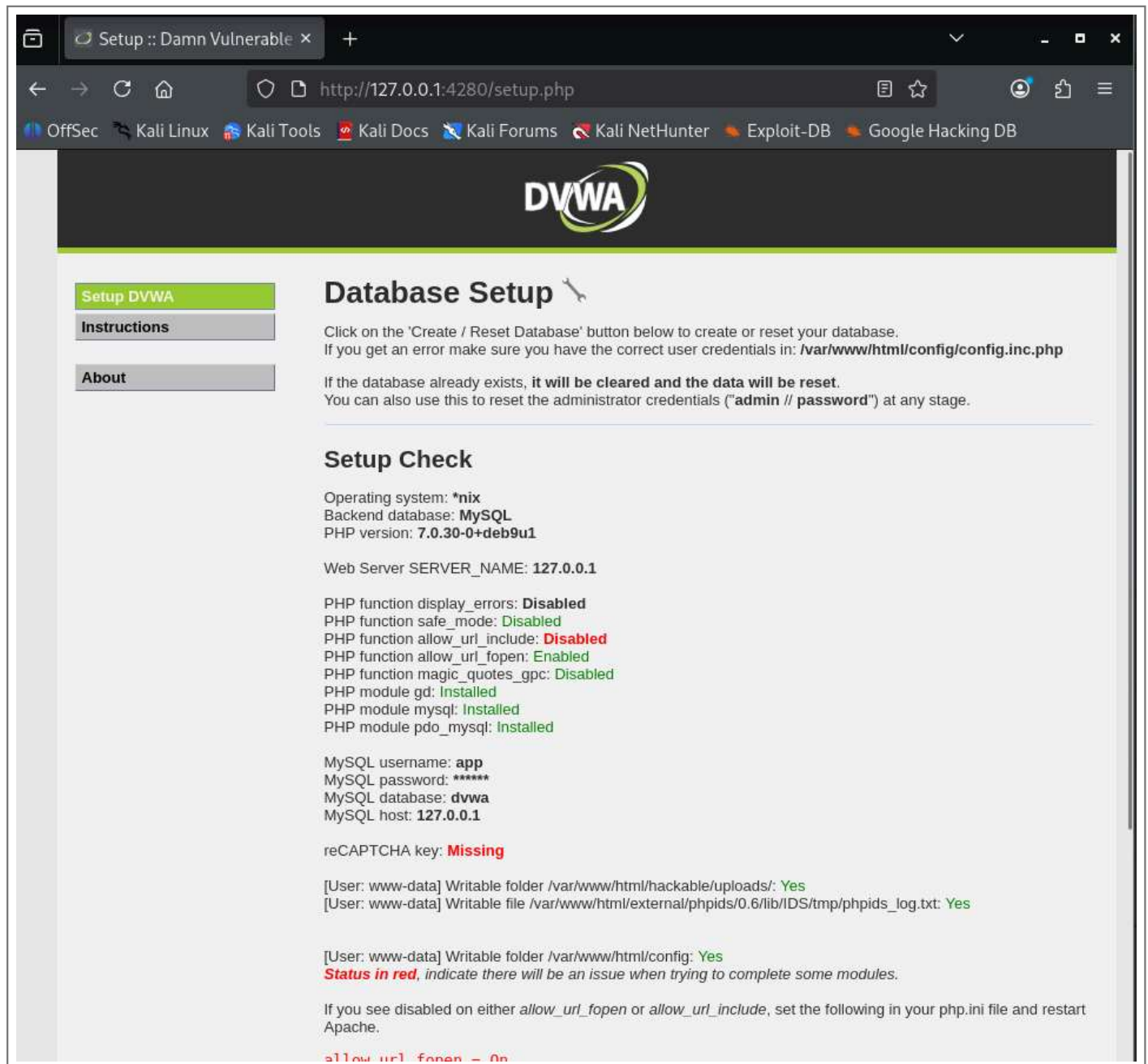


Figure 1: Successful DVWA Setup and Database Configuration

4. HTTP Service Scanning Using Metasploit

Metasploit Framework was used to perform HTTP service scanning against the locally hosted DVWA application.

The following Metasploit module was used:

auxiliary/scanner/http/http_version

Parameter	Value
RHOSTS	127.0.0.1
RPORT	4280

Result: The HTTP service running on port 4280 was successfully identified.

```

interact with a module by name or index. For example info 358, use 358 or use auxiliary
sf > use auxiliary/scanner/http/http_version
sf auxiliary(scanner/http/http_version) > set RHOSTS 127.0.0.1
RHOSTS => 127.0.0.1
sf auxiliary(scanner/http/http_version) > set RPORT 4280
RPORT => 4280
sf auxiliary(scanner/http/http_version) > SHOW OPTIONS
-] Unknown command: SHOW. Did you mean show? Run the help command for more details.
sf auxiliary(scanner/http/http_version) > show options

Module options (auxiliary/scanner/http/http_version):

  Name      Current Setting  Required  Description
  ---      -
Proxies      no              A proxy chain of format type:host:port[,type:host:port]*
RHOSTS      127.0.0.1        yes       The target host(s), see https://docs.metasploit.com/docs/basics/using-metasploit.html
RPORT       4280             yes       The target port (TCP)
SSL         false            no        Negotiate SSL/TLS for outgoing connections
THREADS     1                yes       The number of concurrent threads (max one per host)
VHOST       no               HTTP server virtual host

View the full module info with the info, or info -d command.
sf auxiliary(scanner/http/http_version) > run
[*] 127.0.0.1:4280 Apache/2.4.25 (Debian) ( 302-login.php )
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
sf auxiliary(scanner/http/http_version) >

```

Figure 2: HTTP Service Scanning Using Metasploit

5. Directory Enumeration Using Metasploit

Directory enumeration was performed using the Metasploit module:

auxiliary/scanner/http/dir_scanner

The target application was scanned using the following configuration:

Parameter	Value
RHOSTS	127.0.0.1
RPORT	4280

Directories Identified:

- /config/ - HTTP 200
- /docs/ - HTTP 200
- /external/ - HTTP 200
- /icons/ - HTTP 403

The directories **/config/**, **/docs/**, and **/external/** were accessible and returned HTTP status code 200.

The **/icons/** directory returned HTTP status code 403, indicating that access was restricted.

```

msf auxiliary(scanner/http/http_version) > use auxiliary/scanner/http/dir_scanner
msf auxiliary(scanner/http/dir_scanner) > set RHOSTS 127.0.0.1
RHOSTS => 127.0.0.1
msf auxiliary(scanner/http/dir_scanner) > set RPORT 4280
RPORT => 4280
msf auxiliary(scanner/http/dir_scanner) > show options
Module options (auxiliary/scanner/http/dir_scanner):


| Name       | Current Setting                                         | Required | Description                                                                                                          |
|------------|---------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------|
| DICTIONARY | /usr/share/metasploit-framework/data/wmap/wmap_dirs.txt | no       | Path of word dictionary to use                                                                                       |
| PATH       | /                                                       | yes      | The path to identify files                                                                                           |
| Proxies    |                                                         | no       | A proxy chain of format type:host:port[,type:host:port][...] Supported proxies: sapni, socks4, socks5, socks5h, http |
| RHOSTS     | 127.0.0.1                                               | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html               |
| RPORT      | 4280                                                    | yes      | The target port (TCP)                                                                                                |
| SSL        | false                                                   | no       | Negotiate SSL/TLS for outgoing connections                                                                           |
| THREADS    | 1                                                       | yes      | The number of concurrent threads (max one per host)                                                                  |
| VHOST      |                                                         | no       | HTTP server virtual host                                                                                             |


View the full module info with the info, or info -d command.
msf auxiliary(scanner/http/dir_scanner) > run
[*] Detecting error code
[*] Using code '404' as not found for 127.0.0.1
[*] Found http://127.0.0.1:4280/config/ 200 (127.0.0.1)
[*] Found http://127.0.0.1:4280/docs/ 200 (127.0.0.1)
[*] Found http://127.0.0.1:4280/external/ 200 (127.0.0.1)
[*] Found http://127.0.0.1:4280/icons/ 403 (127.0.0.1)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/http/dir_scanner) >

```

Figure 3: Directory Enumeration Results

6. SQL Injection Vulnerability Testing

After completing the scanning and directory enumeration process, the SQL Injection vulnerability module available in DVWA was selected for testing.

6.1 Normal Input Test

A normal User ID value was entered into the application. The application returned information associated with the requested user.

Observation: Normal input returned a single user record as expected.

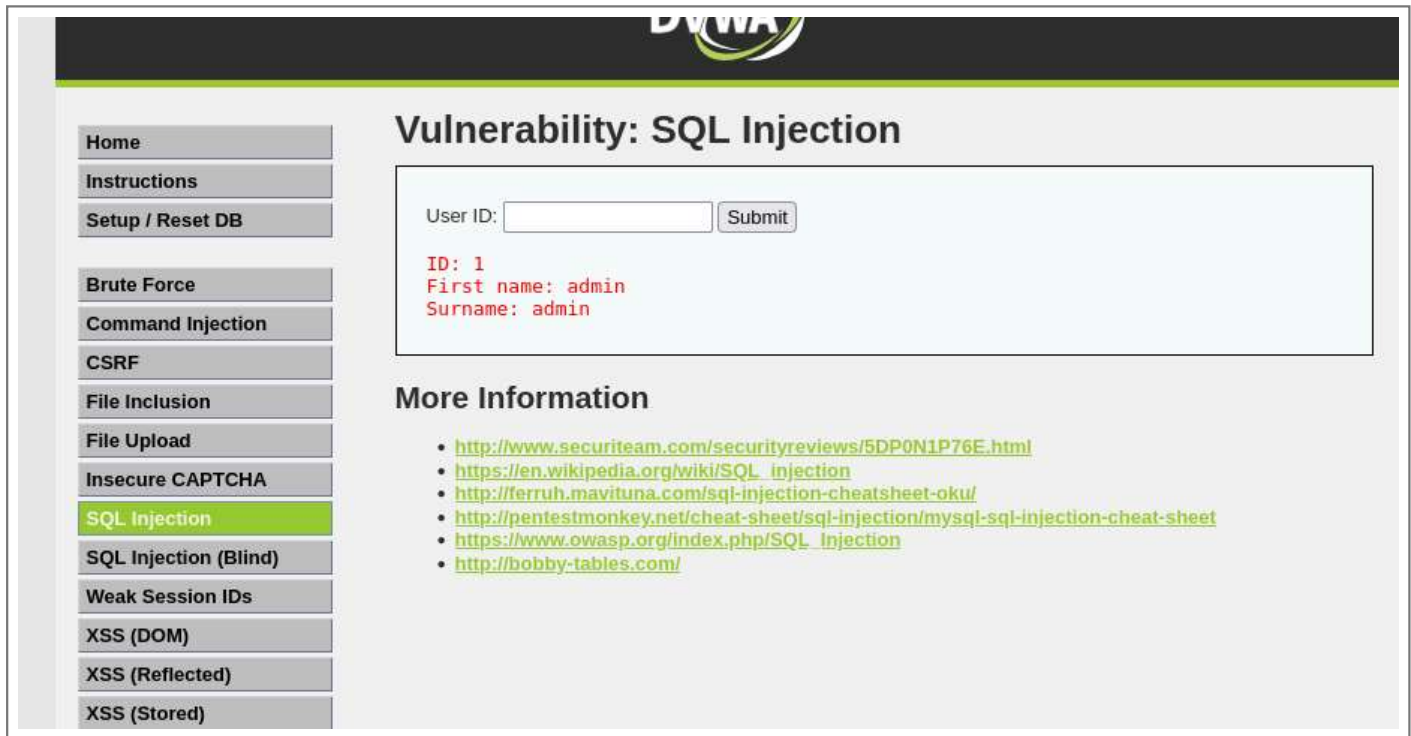


Figure 4: Normal User ID Input

6.2 SQL Injection Test Result

A SQL Injection test payload was entered into the User ID field of the intentionally vulnerable DVWA application.

Test Payload:

' OR '1'='1

Instead of returning information for a single user, the application returned multiple user records.

Vulnerability Identified: SQL Injection

The application accepted the injected input and returned multiple records. This demonstrates that the application was vulnerable to SQL Injection in the tested DVWA environment.

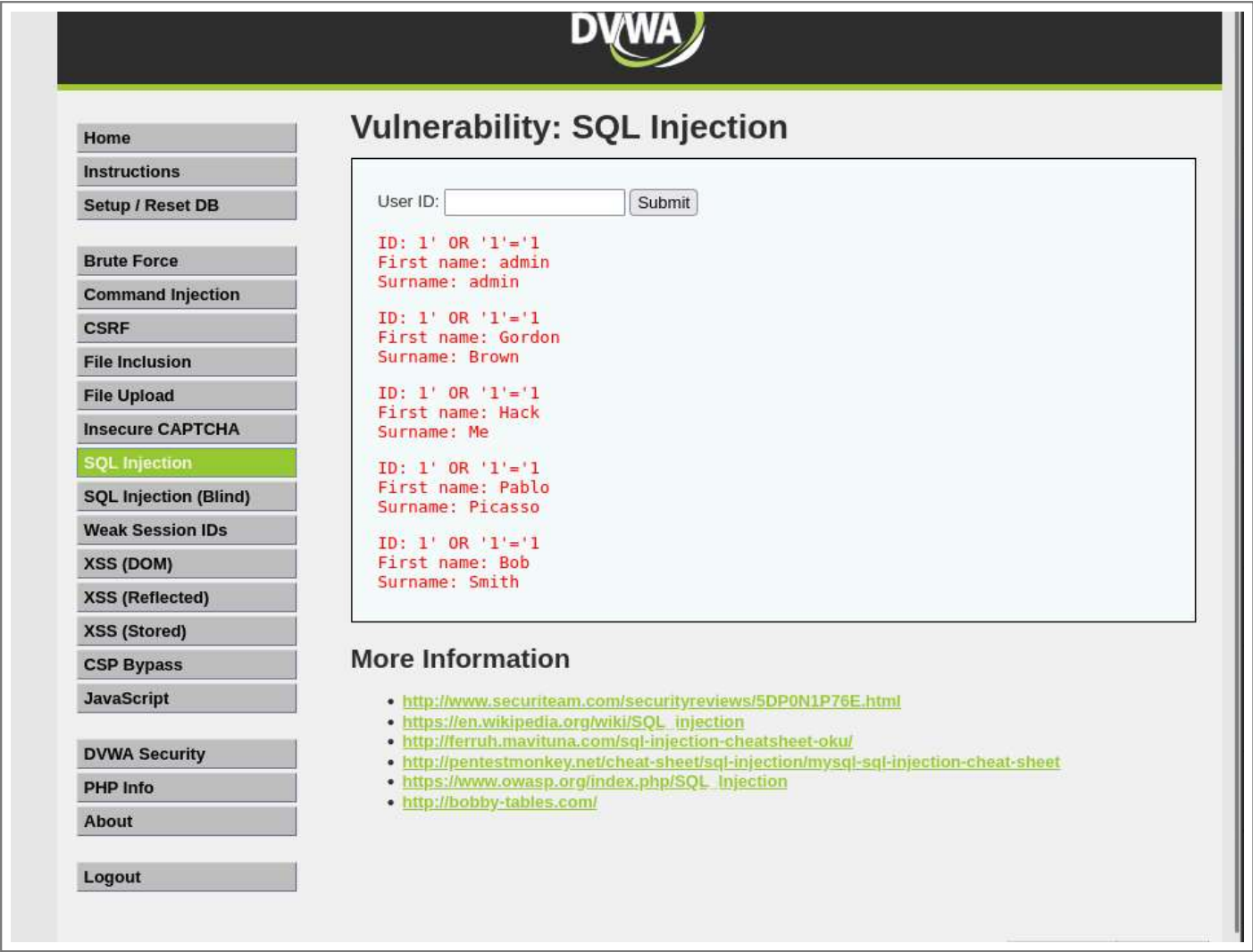


Figure 5: SQL Injection Vulnerability Result

7. Assessment Findings

Finding	Description	Severity
Accessible Directories	Directory enumeration revealed multiple accessible directories including config, docs, and external.	Low
SQL Injection	The application accepted malicious SQL input and returned multiple database records.	High

8. Vulnerability Assessment and CVSS Score

The identified SQL Injection vulnerability was assigned a CVSS score based on its potential impact on confidentiality, integrity, and availability.

Vulnerability Name	SQL Injection
Severity	High
CVSS Version	CVSS v3.1
CVSS Score	8.8 (High)
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

9. Security Impact

SQL Injection vulnerabilities can allow an attacker to manipulate database queries by supplying specially crafted input.

Depending on the application and database permissions, successful SQL Injection may result in:

- Unauthorized access to sensitive data
- Exposure of database records
- Authentication bypass
- Modification of database information
- Potential compromise of application data

10. Recommendations

- Use prepared statements and parameterized SQL queries.
- Validate all user input before processing.
- Apply proper input sanitization.
- Use strong authentication and access controls.
- Avoid displaying detailed database errors to users.
- Restrict access to unnecessary directories.
- Perform regular vulnerability assessments and security testing.

11. Conclusion

The web application vulnerability assessment was successfully completed in a controlled local environment using Kali Linux, Docker, DVWA, and the Metasploit Framework.

The assessment included successful deployment and configuration of DVWA, HTTP service scanning, and directory enumeration. The scanning process identified several accessible directories within the target application.

SQL Injection testing demonstrated that the intentionally vulnerable DVWA application accepted malicious SQL input. Normal input returned a single record, whereas the SQL Injection test returned multiple database records.

Based on the assessment, SQL Injection was identified as the major vulnerability, with a CVSS v3.1 score of **8.8 (High)**. Proper input validation, parameterized queries, and secure coding practices are recommended to prevent similar vulnerabilities.

12. Project Repository

The complete Task 5 project and supporting files are available in the following GitHub repository:

GitHub Repository:

https://github.com/Deepthi452005/NovaShyld-Task_5

Web Application Vulnerability Assessment Report

Testing performed in an authorized and controlled DVWA environment.